Guia de consulta: atributos HTML

Do que decorar ao que só se consulta. Feito para estudar e aplicar no meu portfólio. Atualizado em 10 de outubro de 2026.

Como usar este guia

Analogia: a tag é o eletrodoméstico e os atributos são os botões. O `<input>` é uma batedeira; `type`, `required` e `placeholder` são velocidade, timer e voltagem. Os botões do dia a dia você decora, o resto consulta no manual (a MDN).

Os atributos estão separados em 4 níveis:

Nível	Significado	O que fazer
1	Aparecem em quase todo projeto	Decorar
2	Uso semanal	Conhecer bem
3	Ocasionais	Saber que existem e consultar
4	Raros ou especializados	Só consultar

A marca ★ indica atributos que também servem de ponte para CSS e/ou JS (veja a seção "Ponte com CSS e JS").

O HTML tem mais de 200 nomes de atributos se contarmos globais, específicos de cada tag, eventos e ARIA, e surgem novos todo ano. A fonte definitiva é a MDN, na página "Referência de atributos HTML".

O básico: como um atributo funciona

O atributo fica sempre na tag de abertura, no formato `nome="valor"`:

```
<a href="https://github.com/seu-usuario" target="_blank">Meu GitHub</a>
```

Existem dois tipos:

- **Com valor:** `href`, `src`, `alt`, `class`.
- **Booleanos:** valem pela simples presença, como `required`, `disabled`, `checked`, `hidden` e `open`.

Pegadinha: `required="false"` continua ativando o `required`. Para desligar, apague o atributo.

Eles também se dividem em **globais** (funcionam em qualquer tag: `class`, `id`, `lang`, `title`, `hidden` …) e **específicos** (`href` só vale em `a`, `link`, `area` e `base`).

Regras de ouro: use aspas no valor, escreva o nome em minúsculas, não repita o mesmo atributo na mesma tag e nunca repita um `id` na página.

Colinha: atributos por elemento

Um jeito fácil de fixar é decorar por tag:

Elemento	Atributos mais usados
`<html>`	`lang`
`<meta>`	`charset`, `name`, `content`
`<link>`	`rel`, `href`
`<script>`	`src`, `defer`, `type`
`<a>`	`href`, `target`, `rel`, `download`
`<img>`	`src`, `alt`, `width`, `height`, `loading`
`<form>`	`action`, `method`
`<label>`	`for`
`<input>`	`type`, `name`, `id`, `required`, `placeholder`, `autocomplete`
`<textarea>`	`name`, `id`, `rows`, `required`
`<option>`	`value`, `selected`
`<button>`	`type`, `disabled`
`<th>`	`scope`
`<td>` e `<th>`	`colspan`, `rowspan`
`<ol>`	`start`, `reversed`, `type`
`<time>`	`datetime`
`<details>`	`open`
`<video>`	`src`, `controls`, `poster`
`<iframe>`	`src`, `title`, `loading`

Elemento	Atributos mais usados
Qualquer tag	`class`, `id`, `data-*`, `hidden`, `aria-*`

Trecho simplificado do portfólio usando vários deles:

```
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="css/style.css">
  <script src="js/app.js" defer></script>
</head>
<body>
  <img src="img/foto.jpg" alt="Seu Nome sorrindo" width="200" height="200">
  <a href="https://github.com/seu-usuario" target="_blank" rel="noopener
noreferrer">GitHub</a>

  <form action="#" method="get">
    <label for="email">E-mail</label>
    <input type="email" id="email" name="email" autocomplete="email"
required>
    <button type="submit">Enviar</button>
  </form>
</body>
</html>
```

Nível 1: decore (27 itens)

Atributo	Onde	Para quê
`class` ★	global	Agrupa elementos para CSS/JS (vários, separados por espaço)
`id` ★	global	Identificador único: âncora (`#id`), `label for`, JS
`href` ★	`a`, `link`, `area`, `base`	Destino do link ou do arquivo CSS
`src` ★	`img`, `script`, `iframe`, `video`, `audio`, `source`, `track`, `embed`	Origem do recurso
`alt`	`img`, `area`, `input` image	Texto alternativo
`lang` ★	global (em `html`)	Idioma do conteúdo

Atributo	Onde	Para quê
`charset`	`meta`	Codificação (`UTF-8`)
`name` ★	`meta`, `input`, `select`, `textarea`, `button`, `form`, `iframe`, `details`	Nome do campo enviado ou tipo do meta
`content`	`meta`	Valor do meta
`type` ★	`input`, `button`, `script`, `link`, `source`, `ol`, `embed`	Tipo (o significado muda conforme a tag)
`value` ★	`input`, `button`, `option`, `li`, `data`, `meter`, `progress`	Valor
`for` ★	`label`, `output`	Liga ao `id` do campo
`rel`	`link`, `a`, `area`, `form`	Relação (`stylesheet`, `icon`, `noopener`)
`target`	`a`, `area`, `form`, `base`	Onde abrir (`_blank`, `_self`)
`width`, `height`	`img`, `video`, `canvas`, `iframe`, `embed`, `object`	Dimensões em pixels (número sem unidade)
`placeholder`	`input`, `textarea`	Dica no campo (não substitui `label`)
`required` ★	`input`, `select`, `textarea`	Campo obrigatório
`disabled` ★	`button`, `input`, `select`, `textarea`, `fieldset`, `option`	Desativa o controle
`checked` ★	`input` checkbox/radio	Já vem marcado
`selected` ★	`option`	Já vem selecionada
`action`	`form`	URL que recebe o envio
`method`	`form`	`get` ou `post` (também `dialog`)
`title`	global	Tooltip. Não é confiável para acessibilidade (não aparece no toque nem no teclado)
`colspan`, `rowspan`	`td`, `th`	Célula que ocupa várias colunas/linhas
`scope`	`th`	A quem o cabeçalho se refere (`col`, `row`, `colgroup`, `rowgroup`)

Atributo	Onde	Para quê
`style` ★	global	CSS direto na tag. Conheça, mas evite: prefira um arquivo `.css`
`data-*` ★	global	Dados personalizados (`data-status="ativo"`)

Valores para decorar:

- `type` do `<input>` (são 22): `text` , `email` , `tel` , `url` , `number` , `password` , `search` , `date` , `time` , `datetime-local` , `month` , `week` , `color` , `range` , `file` , `checkbox` , `radio` , `hidden` , `submit` , `reset` , `button` , `image` .
- `name` do `<meta>` : `viewport` , `description` , `theme-color` , `color-scheme` , `robots` , `author` .

Nível 2: comuns (uso semanal)

Atributo	Onde	Para quê
`hidden` ★	global	Esconde o elemento (aceita também `until-found`)
`tabindex` ★	global	`0` entra na ordem do Tab; `-1` só via JS. Evite valores positivos
`defer` ★	`script`	Roda o JS depois de ler o HTML (o mais recomendado)
`async` ★	`script`	Roda assim que baixar, sem garantir ordem
`role` ★	global	Papel ARIA (só quando não existe tag nativa)
`aria-label` ★	global	Nome acessível em texto
`aria-labelledby` ★	global	Nome acessível vindo do `id` de outro elemento
`aria-describedby` ★	global	Descrição extra (ex.: dica de um campo)
`aria-hidden` ★	global	Esconde de leitores de tela (nunca em algo focável)
`aria-current` ★	global	Item atual (`page` , `step` , `true`)

Atributo	Onde	Para quê
`aria-expanded` ★	global	Aberto ou fechado (menus, acordeões)
`loading`	`img`, `iframe`	`lazy` adia o carregamento
`controls`	`video`, `audio`	Mostra os controles do player
`autoplay`, `loop`, `muted`	`video`, `audio`	Toca sozinho / repete / sem som (o navegador costuma exigir `muted` para autoplay)
`poster`	`video`	Imagem exibida antes de tocar
`download`	`a`, `area`	Baixa o arquivo em vez de navegar
`autocomplete`	`form`, `input`, `select`, `textarea`	Preenchimento automático (`name`, `email`, `tel`, `current-password`)
`autofocus`	global	Foca ao carregar (use com moderação)
`readonly` ★	`input`, `textarea`	Visível e enviado, mas não editável
`min`, `max`, `step`	`input`, `meter`, `progress`	Limites e passo de números/datas
`minlength`, `maxlength`	`input`, `textarea`	Limite de caracteres
`pattern` ★	`input`	Validação por expressão regular
`multiple`	`input` (email, file), `select`	Aceita vários valores
`accept`	`input` file	Tipos de arquivo aceitos
`rows`, `cols`	`textarea`	Tamanho em linhas/colunas
`datetime`	`time`, `del`, `ins`	Data/hora legível por máquina
`open` ★	`details`, `dialog`	Já vem aberto
`novalidate`	`form`	Desliga a validação nativa
`srcset`, `sizes`	`img`, `source`	Imagens responsivas
`media`	`link`, `source`, `style`	Condição de mídia (`(min-width: 600px)`)

Atributo	Onde	Para quê
`http-equiv`	`meta`	Equivale a um cabeçalho HTTP (`refresh`, `content-security-policy`)
`dir` ★	global	Direção do texto (`ltr`, `rtl`, `auto`)

Nível 3: ocasionais (consulte quando precisar)

Atributo	Onde	Para quê
`crossorigin`	`img`, `script`, `link`, `audio`, `video`	Requisição CORS (`anonymous`, `use-credentials`)
`integrity`	`script`, `link`	Hash de segurança (SRI) para recursos de CDN
`referrerpolicy`	`a`, `area`, `img`, `iframe`, `link`, `script`	Quanto do endereço de origem é enviado
`form`	`input`, `button`, `select`, `textarea`, `fieldset`, `output`, `object`	Associa o controle a um `<form>` pelo `id`, mesmo fora dele
`list`	`input`	Liga a um `<datalist>`
`enctype`	`form`	Codificação do envio (`multipart/form-data` para arquivos)
`formaction`, `formmethod`, `formenctype`, `formnovalidate`, `formtarget`	`button`, `input` submit/image	Sobrescrevem as configs do `form` naquele botão
`label`	`option`, `optgroup`, `track`	Rótulo alternativo
`start`, `reversed`	`ol`	Número inicial / contagem decrescente
`headers`	`td`, `th`	IDs dos cabeçalhos (tabelas complexas)
`abbr`	`th`	Versão curta do cabeçalho

Atributo	Onde	Para quê
`span`	`col`, `colgroup`	Quantas colunas abrange
`cite`	`blockquote`, `q`, `del`, `ins`	URL da fonte
`hreflang`	`a`, `link`	Idioma do destino
`sandbox`	`iframe`	Restringe as permissões do conteúdo incorporado
`allow`, `allowfullscreen`	`iframe`	Permissões (câmera, tela cheia)
`srcdoc`	`iframe`	HTML escrito direto no atributo
`preload`	`video`, `audio`	`none`, `metadata` ou `auto`
`playsinline`	`video`	Toca na própria página no iOS
`kind`, `srclang`, `default`	`track`	Legendas e descrições (`subtitles`, `captions`)
`contenteditable` ★	global	Torna o elemento editável (`true`, `false`, `plaintext-only`)
`draggable` ★	global	Permite arrastar
`spellcheck`	global	Corretor ortográfico
`translate`	global	`no` impede a tradução automática (ex.: nomes de marcas)
`inputmode`	global	Teclado virtual (`numeric`, `decimal`, `tel`, `email`, `url`, `search`, `text`, `none`)
`enterkeyhint`	global	Rótulo da tecla Enter no celular (`go`, `next`, `search`, `send`)
`autocapitalize`	global	Capitalização automática (`none`, `sentences`, `words`, `characters`)

Atributo	Onde	Para quê
`autocorrect`	global	Correção automática (`on` ou `off`)
`popover` ★	global	Torna o elemento um popover (`auto` , `manual`)
`popovertarget` , `popovertargetaction` ★	`button` , `input`	Botão que abre/fecha o popover pelo `id` (`show` , `hide` , `toggle`)
`inert` ★	global	Trecho "morto": sem foco, clique ou leitura
`decoding`	`img`	`async` , `sync` ou `auto`
`fetchpriority`	`img` , `link` , `script`	`high` , `low` ou `auto`
`wrap`	`textarea`	`hard` ou `soft`
`size`	`input` , `select`	Largura em caracteres / itens visíveis (prefira CSS)
`dirname`	`input` , `textarea`	Envia a direção do texto junto
`capture`	`input` file	Usa câmera/microfone do celular
`low` , `high` , `optimum`	`meter`	Faixas do medidor
`data`	`object`	URL do recurso
`usemap` , `ismap`	`img`	Mapa de imagem clicável
`coords` , `shape` (e `name` no `map`)	`area`	Áreas clicáveis (`rect` , `circle` , `poly` , `default`)
`as`	`link`	Tipo do recurso em `rel="preload"`
`nonce`	global	Token para política de segurança (CSP)
`nomodule`	`script`	Só roda em navegadores sem suporte a módulos
`blocking`	`link` , `script` , `style`	`render` bloqueia a renderização até carregar

Atributo	Onde	Para quê
`accesskey`	global	Atalho de teclado (conflita com atalhos do sistema; evite)

Nível 4: raros ou especializados

Atributo	Onde	Para quê
`accept-charset`	`form`	Codificações aceitas no envio
`imagesrcset`, `imagesizes`	`link`	`srcset` / `sizes` para `rel="preload" as="image"`
`itemscope`, `itemtype`, `itemprop`, `itemid`, `itemref`	global	Microdados (dados estruturados para buscadores)
`slot`	global	Posição em Web Components
`is`	global	Cria elemento nativo customizado
`part`, `exportparts`	global	Expõe partes do Shadow DOM para `::part()`
`shadowrootmode` e variantes (`shadowrootdelegatesfocus`, `shadowrootclonable`, `shadowrootserializable`)	`template`	Declara Shadow DOM direto no HTML
`command`, `commandfor`	`button`	Botão que aciona outro elemento sem JS (suporte ainda crescendo)
`closedby`	`dialog`	Como o diálogo pode ser fechado (suporte ainda crescendo)
`writingsuggestions`	global	Liga/desliga sugestões de escrita
`xmlns`, `xml:lang`	`html`	Só para XHTML/XML
`controlslist`, `disablepictureinpicture`, `disableremoteplayback`	`video`, `audio`	Ajustes do player (suporte varia)

Atributo	Onde	Para quê
`elementtiming`, `csp`	`img`, `iframe`	Performance / CSP (experimentais)
`webkitdirectory`	`input` file	Seleção de pastas (não padrão, mas bem suportado)
`ping`	`a`, `area`	Rastreio de cliques (Firefox desativa por padrão)

Ponte com CSS e JS

Os atributos marcados com ★ são a "cola" entre as três linguagens.

Atributo	No CSS	No JS
`class`	`.projeto { }`	`el.classList.add('ativo')`, `remove`, `toggle`
`id`	`#sobre { }` e `:target`	`document.getElementById('sobre')`
`style`	Inline (peso altíssimo, difícil de sobrescrever)	`el.style.color = 'red'`
`data-*`	`[data-status="ativo"]`	`el.dataset.status` (`data-meu-dado` vira `dataset.meuDado`)
`hidden`	Vira `display: none`, mas um `display:` seu no CSS sobrescreve	`el.hidden = true`
`href`	`a[href^="https"]`, `:link`, `:visited`	`a.href`, `location.hash`
`name`, `value`, `type`	`input[type="email"]`	`new FormData(form)`, `input.value`
`required`, `disabled`, `checked`, `readonly`, `selected`, `pattern`, `min`, `max`	`:required`, `:disabled`, `:checked`, `:read-only`, `:invalid`, `:valid`, `:in-range`	`.required`, `.disabled`, `.checked`, `input.checkValidity()`
`open`	`details[open]`	`details.open` e o evento `toggle`
`tabindex`, `autofocus`, `inert`	`:focus`, `:focus-visible`	`el.focus()`, `el.inert = true`

Atributo	No CSS	No JS
`popover`, `popovertarget`	`[popover]`, `:popover-open`	`el.showPopover()`, `hidePopover()`
`role`, `aria-*`	`[aria-current="page"]`, `[aria-expanded="true"]`	`setAttribute('aria-expanded', 'true')`
`lang`, `dir`	`:lang(pt)`, `:dir(rtl)`	`document.documentElement.lang`
`width`, `height` (em `img`)	O navegador calcula a proporção e evita o "pulo" da página	`img.naturalWidth`
`<link rel="stylesheet">`	**Porta de entrada do CSS**	
`<script src defer>`		**Porta de entrada do JS**

Exemplo aplicado ao portfólio:

```
<article class="projeto" id="projeto-html" data-status="em-andamento">
  <h3>Página para ensinar HTML</h3>
</article>
```

```
/* CSS seleciona pelo atributo */
.projeto[data-status="em-andamento"] { border-left: 4px solid orange; }
a[href^="https"] { text-decoration: underline; }
nav a[aria-current="page"] { font-weight: bold; }
```

```
// JS lê e altera atributos
const projeto = document.querySelector('#projeto-html');
console.log(projeto.dataset.status);   // "em-andamento"
projeto.dataset.status = 'concluido';  // o CSS acima reage sozinho
projeto.setAttribute('aria-label', 'Projeto');
projeto.toggleAttribute('hidden');     // esconde ou mostra
```

Atributo não é o mesmo que propriedade: `value="abc"` no HTML é o valor *inicial*. O que a pessoa digitou depois vive na propriedade `input.value`, e `getAttribute('value')` continua devolvendo o original.

Atributos de evento (on...)

Ligam JS direto na tag (`<button onclick="...">`), mas o recomendado é `addEventListener('click', ...)` , usando o nome sem o `on` . Você só precisa reconhecê-los.

- **Mouse, ponteiro e toque:** `onclick` , `ondblclick` , `onauxclick` , `oncontextmenu` , `onmousedown` , `onmouseup` , `onmousemove` , `onmouseenter` , `onmouseleave` , `onmouseover` , `onmouseout` , `onwheel` , `onpointerdown` , `onpointerup` , `onpointermove` , `onpointerenter` , `onpointerleave` , `onpointerover` , `onpointerout` , `onpointercancel` , `ongotpointercapture` , `onlostpointercapture` , `ontouchstart` , `ontouchmove` , `ontouchend` , `ontouchcancel`
- **Teclado:** `onkeydown` , `onkeyup` , `onkeypress` (obsoleto)
- **Foco e formulário:** `onfocus` , `onblur` , `onchange` , `oninput` , `onbeforeinput` , `oninvalid` , `onselect` , `onsubmit` , `onreset` , `onformdata` , `oncopy` , `oncut` , `onpaste`
- **Arrastar:** `ondrag` , `ondragstart` , `ondragend` , `ondragenter` , `ondragleave` , `ondragover` , `ondrop`
- **Mídia:** `onplay` , `onpause` , `onplaying` , `onended` , `onwaiting` , `onseeking` , `onseeked` , `ontimeupdate` , `onvolumechange` , `onratechange` , `ondurationchange` , `onloadstart` , `onloadeddata` , `onloadedmetadata` , `oncanplay` , `oncanplaythrough` , `onprogress` , `onstalled` , `onsuspend` , `onemptied` , `onabort` , `oncuechange`
- **Elemento e página:** `onload` , `onerror` , `onscroll` , `onscrollend` , `onresize` , `ontoggle` , `onbeforetoggle` , `oncancel` , `onclose` , `onslotchange` , `onbeforematch` , `oncontextlost` , `oncontextrestored` , `onsecuritypolicyviolation`
- **Janela (usados no `<body>`):** `onafterprint` , `onbeforeprint` , `onbeforeunload` , `onhashchange` , `onlanguagechange` , `onmessage` , `onmessageerror` , `onoffline` , `ononline` , `onpagehide` , `onpageshow` , `onpopstate` , `onrejectionhandled` , `onstorage` , `onunhandledrejection` , `onunload`

ARIA (acessibilidade)

Regra número 1: se existe uma tag nativa, use a tag. ARIA só entra quando o HTML não basta.

Tem dois tipos: `role="..."` e os atributos `aria-*` . Lista completa dos `aria-*` :

`aria-activedescendant` , `aria-atomic` , `aria-autocomplete` , `aria-braillelabel` , `aria-brailleroledescription` , `aria-busy` , `aria-checked` , `aria-colcount` , `aria-colindex` , `aria-colindextext` , `aria-colspan` , `aria-controls` , `aria-current` , `aria-describedby` , `aria-description` , `aria-details` , `aria-disabled` , `aria-errormessage` , `aria-expanded` , `aria-flowto` , `aria-haspopup` , `aria-hidden` , `aria-invalid` , `aria-`

`keyshortcuts`, `aria-label`, `aria-labelledby`, `aria-level`, `aria-live`, `aria-modal`, `aria-multiline`, `aria-multiselectable`, `aria-orientation`, `aria-owns`, `aria-placeholder`, `aria-posinset`, `aria-pressed`, `aria-readonly`, `aria-relevant`, `aria-required`, `aria-roledescription`, `aria-rowcount`, `aria-rowindex`, `aria-rowindextext`, `aria-rowspan`, `aria-selected`, `aria-setsize`, `aria-sort`, `aria-valuemax`, `aria-valuemin`, `aria-valuenow`, `aria-valuetext`

Obsoletos: `aria-dropeffect` e `aria-grabbed`.

No dia a dia você só precisa de `aria-label`, `aria-labelledby`, `aria-describedby`, `aria-hidden`, `aria-current` e `aria-expanded`, e de `aria-live` quando começar com JS.

Obsoletos (não use, só reconheça em código antigo)

Atributo(s)	Onde	Troque por
`align`, `valign`	`div`, `p`, `h1` – `h6`, `table`, `img`, `td`	CSS (`text-align`, `vertical-align`, Flexbox)
`bgcolor`, `background`	`body`, `table`, `td`	CSS `background`
`text`, `link`, `vlink`, `alink`	`body`	CSS `color`, `a:link`, `a:visited`, `a:active`
`color`, `face`, `size`	`font`	CSS `color`, `font-family`, `font-size`
`size`, `noshade`, `width`	`hr`	CSS
`cellpadding`, `cellspacing`, `frame`, `rules`	`table`	CSS `padding`, `border-spacing`, `border`
`border`	`img`, `object`	CSS `border`
`border`	`table`	Só `border=""` ou `border="1"` passa no validador. Ajuda a enxergar as linhas sem CSS, mas troque depois
`width`, `height`	`table`, `td`, `th`	CSS
`hspace`, `vspace`	`img`, `object`	CSS `margin`
`nowrap`	`td`, `th`	CSS `white-space: nowrap`
`clear`	`br`	CSS `clear`

Atributo(s)	Onde	Troque por
`compact`	`ul`, `ol`, `dl`	CSS
`type`	`ul`, `li`	CSS `list-style-type` (em `ol` o `type` continua válido)
`leftmargin`, `topmargin`, `rightmargin`, `bottommargin`, `marginwidth`, `marginheight`	`body`, `iframe`	CSS `margin`
`char`, `charoff`	`td`, `tr`, `col`	CSS
`frameborder`, `scrolling`, `allowtransparency`	`iframe`	CSS `border`, `overflow`
`longdesc`	`img`, `iframe`	`figure` + `figcaption`, `aria-describedby`
`name`	`a`, `img`	`id`
`charset`, `rev`	`a`, `link`	`<meta charset>` / `rel`
`language`	`script`	`type` ou nada
`event`, `for`	`script`	`addEventListener`
`summary`	`table`	`<caption>`
`abbr`, `axis`, `scope`	`td`	`th scope`, `headers`
`archive`, `classid`, `code`, `codebase`, `codetype`, `declare`, `standby`, `valuetype`	`object`, `applet`, `param`	`embed`, `video`, `iframe`
`datasrc`, `datafld`, `dataformatas`	vários	JS
`profile`, `version`, `manifest`, `scheme`	`head`, `html`, `meta`	Nada
`contextmenu`, `dropzone`, `scoped`, `seamless`, `pubdate`, `keytype`, `challenge`	vários	Removidos da especificação
`autosave`, `results`, `incremental`, `x-webkit-speech`	`input`	Não padronizados
`onkeypress`	global	Evento `keydown`
`http-equiv="X-UA-Compatible"`	`meta`	Era para o Internet Explorer

Válidos, mas desnecessários

O que	Por que dispensar
`type="text/css"` em `link` / `style` e `type="text/javascript"` em `script`	São os padrões
`type="text"` em `input`	É o padrão (valor desconhecido também vira `text`)
`method="get"`, `enctype` padrão, `autocomplete="on"`	São os padrões
`role="navigation"` em `nav`, `role="main"` em `main`, `role="button"` em `button`	O papel já vem da tag
`tabindex="0"` em `a`, `button`, `input`	Já são focáveis
`tabindex` positivo (`1`, `2`...)	Quebra a ordem natural do foco
`rel="noopener"` com `target="_blank"`	Navegadores modernos já aplicam. Manter não faz mal
`async` junto com `defer`	O `async` vence nos navegadores atuais
`draggable="true"` em `img` e `a[href]`	Já é o padrão
`<meta name="keywords">`	O Google ignora
`charset` em `script`	Só aceita `utf-8`, que já é o padrão
`xmlns` em `html`	Só faz sentido em XHTML
`href="#"` ou `href="javascript:void(0)"`	Para ações, use `<button>`
`title` repetindo o texto do elemento	Redundante; leitores de tela podem ler duas vezes
`style` espalhado pela página	Dificulta a manutenção; use classes
`placeholder` no lugar de `label`	Some ao digitar e é pouco acessível
`aria-hidden="true"` em algo focável	Cria "foco fantasma" para leitores de tela
`id` repetido	Inválido; quebra `label for`, âncoras e JS

O que realmente memorizar

1. **Nível 1 inteiro (27 itens):** é o que você vai digitar sem consultar.

2. **Do Nível 2:** `hidden`, `tabindex`, `defer`, `loading`, `autocomplete`, `min` / `max`, `pattern`, `aria-label`, `aria-current` e `aria-expanded`.
3. **Para CSS e JS:** `class`, `id`, `data-*`, `hidden`, `name` / `value` / `type`, `aria-*` e `defer`.
4. **O resto:** saber que existe e consultar na MDN. Nenhum profissional decora o Nível 4.

Treino no portfólio

1. Abra o `index.html` e confira o `<head>`: ele tem `lang`, `charset`, `viewport` e `<title>`?
2. Confira cada `<img>`: tem `alt`, `width` e `height`?
3. Confira cada link externo: tem texto descritivo e `rel="noopener noreferrer"` quando usa `target="_blank"`?
4. Cada `<input>` tem `<label for>` com o mesmo `id`, e o `type` certo?
5. Ligue o CSS com `<link rel="stylesheet" href="css/style.css">` e estilize um projeto usando `class` e `data-status`.
6. Rode o validador (validator.w3.org/nu) para achar atributos errados, repetidos ou obsoletos.